MODUL PRAKTIKUM
INTERNET OF THINGS (IoT)

Menggunakan Simulator Wokwi

Septian Geges, S.Kom., M.Kom.
Efrans Christian, ST., MT.

4 Modul Praktikum Lengkap

Modul 1: Pengenalan Arduino & Kontrol GPIO
Modul 2: Sensor & Aktuator dalam IoT
Modul 3: Konektivitas WiFi & Protokol IoT (MQTT)
Modul 4: Proyek Akhir - Smart Home Monitoring

Program Studi Teknik Informatika / Sistem Informasi
Semester Genap 2025/2026

PETUNJUK UMUM PRAKTIKUM

Tentang Simulator Wokwi

Wokwi (https://wokwi.com) adalah simulator elektronik dan IoT berbasis web yang gratis dan open-source friendly. Wokwi mendukung simulasi Arduino, ESP32, ESP8266, dan berbagai komponen elektronik tanpa memerlukan hardware fisik.

Keunggulan Wokwi Simulator
✓ Gratis dan dapat diakses langsung dari browser tanpa instalasi
✓ Mendukung Arduino Uno, Mega, Nano, ESP32, ESP8266, Raspberry Pi Pico ✓ Tersedia ratusan komponen: sensor, display, motor, komunikasi
✓ Dilengkapi editor kode dengan fitur syntax highlighting ✓ Dapat berbagi project dengan URL (share link)
✓ Mendukung library Arduino populer (DHT, LiquidCrystal, WiFi, MQTT, dsb) ✓ Serial Monitor tersedia untuk debugging

Cara Memulai Wokwi

1. Buka browser dan akses https://wokwi.com
2. Klik tombol 'New Project' atau pilih template yang tersedia
3. Pilih platform (Arduino Uno / ESP32, dst) sesuai modul
4. Tambahkan komponen dari panel sebelah kiri dengan klik '+' atau drag-drop
5. Sambungkan kabel dengan klik pada pin komponen
6. Tulis atau paste kode pada editor di kanan
7. Klik tombol Play (▶) untuk menjalankan simulasi

Alat Alternatif (Opsional)

Simulator	Keterangan
Wokwi (Utama)	Wajib digunakan - berbasis web, gratis, tidak perlu instalasi
TinkerCAD Circuits	Alternatif untuk Arduino, tersedia di tinkercad.com
Fritzing	Software desktop untuk desain sirkuit (perlu instalasi)
PlatformIO + VSCode	IDE desktop untuk development IoT profesional

Tata Tertib Praktikum

- Setiap mahasiswa wajib mengerjakan tugas menggunakan akun Wokwi masing-masing

- Screenshot hasil simulasi wajib disertakan dalam laporan
- Laporan dikumpulkan dalam format PDF paling lambat 1 minggu setelah praktikum
- Dilarang menyalin kode tanpa memahami cara kerja program
- Tugas pendahuluan dikerjakan sebelum praktikum dimulai

MODUL 1
PENGENALAN ARDUINO & KONTROL GPIO

A. Tujuan Praktikum

- Mahasiswa memahami konsep dasar platform Arduino dan ekosistem IoT
- Mahasiswa mampu menggunakan simulator Wokwi untuk simulasi rangkaian
- Mahasiswa dapat memprogram kontrol GPIO (input/output digital dan analog)
- Mahasiswa mampu membuat program kendali LED dan pembacaan tombol (push button)

B. Dasar Teori

1. Arduino dalam Konteks IoT

Arduino adalah platform open-source berbasis mikrokontroler yang digunakan secara luas dalam pengembangan prototipe IoT. Arduino menggunakan bahasa pemrograman berbasis C/C++ yang disederhanakan dan memiliki ekosistem library yang sangat besar.

Dalam konteks IoT, Arduino berperan sebagai edge device yang membaca data dari sensor (suhu, kelembaban, cahaya, gerak, dll) dan mengirimkan data tersebut ke cloud atau gateway. Arduino Uno menggunakan mikrokontroler ATmega328P dengan frekuensi clock 16 MHz, 32KB Flash, 2KB SRAM, dan 14 pin digital I/O.

2. GPIO (General Purpose Input/Output)

GPIO adalah pin serbaguna pada mikrokontroler yang dapat dikonfigurasi sebagai input atau output. Fungsi utama GPIO dalam IoT meliputi:

- Digital Output: Mengontrol LED, relay, buzzer, motor
- Digital Input: Membaca status tombol, sensor digital (PIR, limit switch)
- Analog Input (ADC): Membaca nilai sensor analog (LDR, potensiometer, sensor suhu NTC)
- PWM Output: Mengatur kecepatan motor, kecerahan LED, servo

Spesifikasi Pin Arduino Uno
Pin Digital: D0-D13 (14 pin) — D0,D1 digunakan untuk Serial (TX/RX)
Pin Analog: A0-A5 (6 pin, resolusi 10-bit = nilai 0-1023)
Pin PWM: D3, D5, D6, D9, D10, D11 (ditandai dengan simbol ~) Tegangan Operasi: 5V (input logika HIGH ≥ 3V, LOW ≤ 1.5V)
Arus Max Per Pin: 40mA (total max 200mA)
Tegangan Input: 7-12V (pin VIN) atau 5V (pin 5V)

3. Rangkaian LED dengan Resistor

LED (Light Emitting Diode) memerlukan resistor seri untuk membatasi arus agar tidak terbakar. Nilai resistor dihitung menggunakan Hukum Ohm: R = (Vsupply - VLED) / ILED

Untuk LED standar: Vsupply = 5V, VLED = 2V (merah/kuning) atau 3.2V (biru/putih), ILED = 20mA. Maka: R = (5-2) / 0.02 = 150 Ohm. Nilai standar yang digunakan: 220 Ohm atau 330 Ohm.

C. Alat dan Bahan (Simulator Wokwi)

Komponen	Spesifikasi / Jumlah
Arduino Uno	1 buah (platform utama)
LED Merah	3 buah
LED Kuning	2 buah
LED Hijau	2 buah
Resistor	220 Ohm, 7 buah
Push Button	2 buah
Potensiometer	1 buah (10K Ohm)
Kabel Jumper	Sesuai kebutuhan

D. Percobaan 1: Blink LED Sederhana

Langkah Kerja

8. Buka Wokwi (wokwi.com), pilih 'New Project' > 'Arduino Uno'
9. Tambahkan komponen LED merah dari panel komponen (+)
10. Tambahkan Resistor 220 Ohm
11. Sambungkan: Pin D13 Arduino → Resistor 220Ω → Anoda LED → GND Arduino → Katoda LED
12. Salin kode berikut ke editor Wokwi:

```
// Modul 1 - Percobaan 1: Blink LED
// Platform: Arduino Uno | Simulator: Wokwi

#define LED_PIN 13    // Pin LED terhubung ke D13

void setup() {
  pinMode(LED_PIN, OUTPUT);  // Konfigurasi D13 sebagai output
  Serial.begin(9600);        // Inisialisasi Serial Monitor
  Serial.println("=== Blink LED Program Started ===");
}

void loop() {
  digitalWrite(LED_PIN, HIGH);  // Nyalakan LED
  Serial.println("LED ON");
```

```
  delay(1000);                  // Tunggu 1 detik

  digitalWrite(LED_PIN, LOW);   // Matikan LED
  Serial.println("LED OFF");
  delay(1000);                  // Tunggu 1 detik
}
```

13. Klik tombol Play (▶) dan amati LED berkedip di simulator
14. Buka Serial Monitor untuk melihat output teks

E. Percobaan 2: Traffic Light dengan Tiga LED

Langkah Kerja

15. Tambahkan 3 LED (merah D8, kuning D9, hijau D10) dengan resistor masing-masing
16. Rangkai sesuai diagram: tiap LED seri dengan resistor 220Ω ke GND
17. Upload kode berikut:

```
// Modul 1 - Percobaan 2: Traffic Light

#define LED_MERAH   8
#define LED_KUNING  9
#define LED_HIJAU   10

void setup() {
  pinMode(LED_MERAH,  OUTPUT);
  pinMode(LED_KUNING, OUTPUT);
  pinMode(LED_HIJAU,  OUTPUT);
  Serial.begin(9600);
}

void trafficState(bool r, bool y, bool g, String msg, int dur) {
  digitalWrite(LED_MERAH,  r ? HIGH : LOW);
  digitalWrite(LED_KUNING, y ? HIGH : LOW);
  digitalWrite(LED_HIJAU,  g ? HIGH : LOW);
  Serial.println("Status: " + msg);
  delay(dur);
}

void loop() {
  trafficState(HIGH, LOW, LOW,  "MERAH - BERHENTI",  3000);
  trafficState(LOW,  HIGH, LOW, "KUNING - BERSIAP",  1000);
  trafficState(LOW,  LOW, HIGH, "HIJAU - JALAN",     3000);
}
```

F. Percobaan 3: Kontrol LED dengan Push Button dan Analog Input

```
// Modul 1 - Percobaan 3: Button & Analog

#define BTN_PIN   2     // Push button di D2
#define LED_PIN   13    // LED di D13
#define POT_PIN   A0    // Potensiometer di A0
#define LED_PWM   9     // LED PWM di D9

void setup() {
  pinMode(BTN_PIN, INPUT_PULLUP); // Input dengan pull-up internal
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  // Baca tombol (LOW = ditekan karena INPUT_PULLUP)
  bool btnState = !digitalRead(BTN_PIN);
  digitalWrite(LED_PIN, btnState ? HIGH : LOW);

  // Baca potensiometer dan map ke PWM (0-255)
  int potVal = analogRead(POT_PIN);  // 0 - 1023
  int brightness = map(potVal, 0, 1023, 0, 255);
  analogWrite(LED_PWM, brightness);

  Serial.print("Button: "); Serial.print(btnState ? "DITEKAN" : "BEBAS");
  Serial.print(" | Pot: "); Serial.print(potVal);
  Serial.print(" | Brightness: "); Serial.println(brightness);
  delay(200);
}
```

G. Tugas Pendahuluan

18. Jelaskan perbedaan antara GPIO digital dan analog dalam konteks IoT!
19. Hitung nilai resistor yang tepat untuk LED biru (VLED = 3.2V, ILED = 20mA) dengan supply 5V!
20. Apa fungsi INPUT_PULLUP pada konfigurasi pin? Gambarkan rangkaian ekuivalennya!

H. Tugas Akhir Praktikum

21. Modifikasi percobaan 2 agar tombol dapat mengganti mode: otomatis/manual
22. Buat program 'running LED' dengan 8 LED dari pin D2-D9 menggunakan array
23. Implementasikan Knight Rider effect (pola LED maju-mundur) dengan kecepatan yang diatur oleh potensiometer

Link Template Wokwi Modul 1
Traffic Light Starter: https://wokwi.com/projects/new/arduino-uno Catatan: Setelah membuka link, tambahkan komponen sesuai langkah kerja di atas
Gunakan fitur 'Save' (Ctrl+S) agar project tersimpan di akun Wokwi Anda

MODUL 2
SENSOR & AKTUATOR DALAM IoT

A. Tujuan Praktikum

- Mahasiswa memahami berbagai jenis sensor yang umum digunakan dalam sistem IoT
- Mahasiswa dapat membaca data dari sensor DHT22 (suhu dan kelembaban)
- Mahasiswa dapat menampilkan data sensor pada LCD 16x2 menggunakan I2C
- Mahasiswa memahami konsep threshold dan sistem alarm berbasis sensor

B. Dasar Teori

1. Sensor dalam Sistem IoT

Sensor adalah komponen kunci dalam sistem IoT yang bertugas mengumpulkan data dari lingkungan fisik (dunia nyata) dan mengubahnya menjadi sinyal listrik yang dapat diproses oleh mikrokontroler. Data inilah yang kemudian dikirimkan ke cloud untuk dianalisis.

Jenis Sensor	Contoh Aplikasi IoT
Suhu & Kelembaban (DHT11/22)	Smart farming, HVAC otomatis, weather station
Cahaya (LDR / BH1750)	Smart lighting, greenhouse monitoring
Gerak/Kehadiran (PIR HC-SR501)	Smart security, otomasi lampu ruangan
Jarak (HC-SR04 Ultrasonik)	Smart parking, level air tangki
Gas (MQ-2, MQ-7)	Detektor asap, sistem keamanan gas
Tekanan (BMP280)	Weather station, altimeter
Akselerometer (MPU-6050)	Wearable, deteksi getaran mesin

2. Sensor DHT22

DHT22 (AM2302) adalah sensor suhu dan kelembaban digital dengan protokol komunikasi single-wire. Spesifikasi utama:

- Range suhu: -40°C hingga +80°C, akurasi ±0.5°C
- Range kelembaban: 0-100% RH, akurasi ±2-5% RH
- Resolusi: 0.1°C dan 0.1% RH
- Tegangan operasi: 3.3V - 5.5V
- Waktu sampling: minimal 2 detik per pembacaan

3. LCD 16x2 dengan I2C

LCD 16x2 dapat menampilkan 2 baris teks dengan masing-masing 16 karakter. Modul I2C (PCF8574) mengurangi kebutuhan pin dari 6-8 pin menjadi hanya 2 pin (SDA dan SCL), sangat efisien untuk penggunaan bersamaan dengan sensor lain.

Koneksi I2C pada Arduino Uno

SDA → Pin A4 (Arduino Uno)
SCL → Pin A5 (Arduino Uno) VCC → 5V GND → GND
Alamat I2C Default LCD: 0x27 (beberapa modul menggunakan 0x3F)

C. Alat dan Bahan (Simulator Wokwi)

Komponen	Jumlah
Arduino Uno	1 buah
Sensor DHT22	1 buah
LCD 16x2 + I2C Module	1 buah
Sensor LDR (Photoresistor)	1 buah
Resistor 10K Ohm	1 buah (pull-up DHT)
Resistor 10K Ohm	1 buah (voltage divider LDR)
LED Merah + Resistor 220Ω	1 buah (alarm indicator)
Buzzer	1 buah

D. Percobaan 1: Membaca Sensor DHT22

Konfigurasi Library di Wokwi

Pada Wokwi, library DHT sudah tersedia. Tambahkan file libraries.txt pada project dengan konten:

```
# Tambahkan ke file libraries.txt di Wokwi
DHT sensor library
LiquidCrystal I2C
```

Kode Program

```
// Modul 2 - Percobaan 1: Sensor DHT22
#include <DHT.h>

#define DHT_PIN  2      // Data pin DHT22 ke D2
#define DHT_TYPE DHT22  // Tipe sensor: DHT22

DHT dht(DHT_PIN, DHT_TYPE);

void setup() {
  Serial.begin(9600);
  dht.begin();
  Serial.println("=== DHT22 Sensor Monitor ===");
```

```
  Serial.println("Waktu\t\tSuhu(C)\tKelembaban(%)");
  Serial.println("----------------------------------------");
}

void loop() {
  delay(2000);  // DHT22 butuh minimal 2 detik antar pembacaan

  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature(); // Celcius
  float heatIndex = dht.computeHeatIndex(temperature, humidity, false);

  // Cek apakah pembacaan berhasil
  if (isnan(humidity) || isnan(temperature)) {
    Serial.println("ERROR: Gagal membaca sensor DHT22!");
    return;
  }

  Serial.print(millis()/1000); Serial.print("s\t");
  Serial.print(temperature, 1); Serial.print("°C\t");
  Serial.print(humidity, 1); Serial.print("%\t");
  Serial.print("Heat Index: "); Serial.print(heatIndex, 1);
  Serial.println("°C");
}
```

E. Percobaan 2: Tampilan LCD & Sistem Alarm Suhu

```
// Modul 2 - Percobaan 2: DHT22 + LCD + Alarm
#include <DHT.h>
#include <LiquidCrystal_I2C.h>

#define DHT_PIN    2
#define DHT_TYPE   DHT22
#define BUZZER_PIN 8
#define LED_ALARM  9
#define TEMP_MAX   30.0  // Threshold suhu (°C)
#define HUM_MAX    80.0  // Threshold kelembaban (%)

DHT dht(DHT_PIN, DHT_TYPE);
LiquidCrystal_I2C lcd(0x27, 16, 2); // Alamat 0x27, 16 kolom, 2 baris

void setup() {
  Serial.begin(9600);
  dht.begin();
  lcd.init();
```

```
  lcd.backlight();
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(LED_ALARM, OUTPUT);

  lcd.setCursor(0, 0); lcd.print("  IoT Monitor   ");
  lcd.setCursor(0, 1); lcd.print(" Initializing...");
  delay(2000);
}

void loop() {
  delay(2000);
  float hum  = dht.readHumidity();
  float temp = dht.readTemperature();
  if (isnan(hum) || isnan(temp)) return;

  // Tampilkan di LCD
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Suhu: ");
  lcd.print(temp, 1);
  lcd.print((char)223); // Simbol derajat °
  lcd.print("C");

  lcd.setCursor(0, 1);
  lcd.print("Hum : ");
  lcd.print(hum, 1);
  lcd.print("%");

  // Sistem alarm
  bool alarm = (temp > TEMP_MAX || hum > HUM_MAX);
  digitalWrite(LED_ALARM, alarm);
  if (alarm) {
    tone(BUZZER_PIN, 1000, 300); // Buzzer 1kHz, 300ms
    lcd.setCursor(14, 0);
    lcd.print("!!");
    Serial.println("[ALARM] Kondisi melebihi batas!");
  } else {
    noTone(BUZZER_PIN);
  }

  Serial.print("Suhu:"); Serial.print(temp);
  Serial.print(" Hum:"); Serial.println(hum);
}
```

F. Percobaan 3: Sensor Cahaya (LDR) - Smart Lighting

```
// Modul 2 - Percobaan 3: Smart Lighting dengan LDR

#define LDR_PIN    A0   // LDR di analog A0
#define LED_PIN    9    // LED PWM di D9
#define THRESHOLD  400  // Nilai cahaya ambang (0-1023)

void setup() {
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(9600);
  Serial.println("Smart Lighting System");
}

void loop() {
  int ldrValue = analogRead(LDR_PIN);

  // Semakin gelap → LDR resistance tinggi → nilai ADC tinggi
  // LED menyala otomatis saat gelap
  int ledBrightness = 0;
  if (ldrValue > THRESHOLD) {
    // Map nilai LDR ke brightness LED (inverse)
    ledBrightness = map(ldrValue, THRESHOLD, 1023, 0, 255);
  }
  analogWrite(LED_PIN, ledBrightness);

  Serial.print("LDR: "); Serial.print(ldrValue);
  Serial.print(" | Brightness: "); Serial.print(ledBrightness);
  Serial.println(ldrValue > THRESHOLD ? " [LAMPU ON]" : " [LAMPU OFF]");
  delay(500);
}
```

G. Tugas Pendahuluan

24. Jelaskan perbedaan antara sensor DHT11 dan DHT22 dari segi spesifikasi!
25. Bagaimana cara kerja protokol komunikasi single-wire yang digunakan DHT22?
26. Apa itu I2C? Sebutkan keuntungan menggunakan I2C dibanding parallel interface untuk LCD!

H. Tugas Akhir Praktikum

27. Tambahkan sensor PIR (HC-SR501) untuk mendeteksi kehadiran orang. Integrasikan dengan sistem alarm!
28. Buat sistem logging data: setiap 10 detik, catat nilai suhu dan kelembaban ke Serial Monitor dalam format CSV (timestamp,suhu,kelembaban)

29. Implementasikan rata-rata bergerak (moving average) 5 sampel untuk memperhalus pembacaan sensor

MODUL 3
KONEKTIVITAS WiFi & PROTOKOL IoT (MQTT)

A. Tujuan Praktikum

- Mahasiswa memahami arsitektur IoT berbasis cloud dan protokol komunikasi
- Mahasiswa dapat memprogram ESP32 untuk terhubung ke jaringan WiFi
- Mahasiswa memahami dan mengimplementasikan protokol MQTT (Publish/Subscribe)
- Mahasiswa dapat mengirim dan menerima data sensor melalui broker MQTT publik

B. Dasar Teori

1. ESP32 - Platform IoT Modern

ESP32 adalah mikrokontroler dual-core dengan kemampuan WiFi 802.11 b/g/n dan Bluetooth 4.2/BLE terintegrasi. ESP32 sangat populer dalam pengembangan IoT karena biaya rendah, performa tinggi, dan ekosistem yang matang.

Fitur	Spesifikasi ESP32
Processor	Dual-core Xtensa LX6, 240 MHz
RAM	520 KB SRAM
Flash	4 MB (tergantung modul)
WiFi	802.11 b/g/n, 2.4 GHz
Bluetooth	BT 4.2 + BLE 5.0
GPIO	34 pin digital, 18 ADC, 2 DAC
Protokol	I2C, SPI, UART, I2S, PWM
Security	AES, RSA, SHA2, WPA2

2. Protokol MQTT

MQTT (Message Queuing Telemetry Transport) adalah protokol messaging ringan berbasis publish/subscribe yang dirancang khusus untuk perangkat IoT dengan bandwidth terbatas. MQTT beroperasi di atas TCP/IP dan menggunakan port 1883 (plain) atau 8883 (TLS/SSL).

Tiga komponen utama MQTT: (1) Publisher - perangkat yang mengirimkan data ke broker, (2) Broker - server perantara yang menerima dan mendistribusikan pesan (contoh: HiveMQ, Mosquitto), dan (3) Subscriber - perangkat/aplikasi yang menerima data dari broker.

Broker MQTT Publik untuk Praktikum
HiveMQ Public Broker: broker.hivemq.com (Port 1883)
EMQX Public Broker: broker.emqx.io (Port 1883)
Mosquitto Test: test.mosquitto.org (Port 1883)

PERHATIAN: Broker publik bersifat umum — JANGAN kirim data sensitif!
Format topic yang disarankan: iot-praktikum/[NIM]/[sensor_name]

3. Format Data JSON untuk IoT

JSON (JavaScript Object Notation) adalah format pertukaran data yang ringan dan mudah dibaca. Dalam IoT, JSON digunakan untuk membungkus multiple data sensor dalam satu payload:

```
// Contoh payload JSON untuk data sensor
{
  "device_id": "ESP32-LAB-01",
  "timestamp": 1700000000,
  "sensor": {
    "temperature": 28.5,
    "humidity": 65.2,
    "light": 512
  },
  "status": "normal"
}
```

C. Percobaan 1: ESP32 Koneksi WiFi

Pada Wokwi, pilih 'New Project' > 'ESP32' untuk menggunakan platform ESP32. Wokwi menyediakan simulasi WiFi virtual.

```
// Modul 3 - Percobaan 1: Koneksi WiFi ESP32
#include <WiFi.h>

// Konfigurasi WiFi - Wokwi menyediakan AP simulasi
const char* SSID     = "Wokwi-GUEST";  // SSID default Wokwi
const char* PASSWORD = "";              // Tanpa password

void printWifiStatus() {
  Serial.println("\n=== WiFi Connected ===");
  Serial.print("SSID    : "); Serial.println(WiFi.SSID());
  Serial.print("IP Addr : "); Serial.println(WiFi.localIP());
  Serial.print("Signal  : "); Serial.print(WiFi.RSSI());
  Serial.println(" dBm");
}

void setup() {
  Serial.begin(115200);
  Serial.println("Menghubungkan ke WiFi...");

  WiFi.mode(WIFI_STA);
```

```
  WiFi.begin(SSID, PASSWORD);

  int attempts = 0;
  while (WiFi.status() != WL_CONNECTED && attempts < 20) {
    delay(500);
    Serial.print(".");
    attempts++;
  }

  if (WiFi.status() == WL_CONNECTED) {
    printWifiStatus();
  } else {
    Serial.println("\nGAGAL terhubung ke WiFi!");
  }
}

void loop() {
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("WiFi terputus! Reconnecting...");
    WiFi.reconnect();
  }
  delay(5000);
}
```

D. Percobaan 2: Publish Data Sensor via MQTT

Tambahkan library PubSubClient pada file libraries.txt di Wokwi: tambahkan baris 'PubSubClient'

```
// Modul 3 - Percobaan 2: MQTT Publisher
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>
#include <ArduinoJson.h>

#define DHT_PIN  4
#define DHT_TYPE DHT22

const char* SSID        = "Wokwi-GUEST";
const char* PASSWORD    = "";
const char* MQTT_SERVER = "broker.hivemq.com";
const int   MQTT_PORT   = 1883;
const char* CLIENT_ID   = "esp32-iot-lab-001"; // Ganti dengan NIM
const char* TOPIC_PUB   = "iot-praktikum/12345/sensor"; // Ganti NIM
const char* TOPIC_CMD   = "iot-praktikum/12345/command";
```

```
WiFiClient   wifiClient;
PubSubClient mqtt(wifiClient);
DHT          dht(DHT_PIN, DHT_TYPE);

void mqttCallback(char* topic, byte* payload, unsigned int length) {
  String msg = "";
  for (int i = 0; i < length; i++) msg += (char)payload[i];
  Serial.println("[MQTT IN] " + String(topic) + " : " + msg);
}

void connectMQTT() {
  while (!mqtt.connected()) {
    Serial.print("Connecting to MQTT...");
    if (mqtt.connect(CLIENT_ID)) {
      Serial.println(" connected!");
      mqtt.subscribe(TOPIC_CMD);
    } else {
      Serial.print(" failed (rc=");
      Serial.print(mqtt.state());
      Serial.println("). Retry in 5s...");
      delay(5000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  dht.begin();
  WiFi.begin(SSID, PASSWORD);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println("\nWiFi OK! IP: " + WiFi.localIP().toString());
  mqtt.setServer(MQTT_SERVER, MQTT_PORT);
  mqtt.setCallback(mqttCallback);
}

void loop() {
  if (!mqtt.connected()) connectMQTT();
  mqtt.loop();

  static unsigned long lastMsg = 0;
  if (millis() - lastMsg > 5000) {  // Kirim setiap 5 detik
    lastMsg = millis();
    float temp = dht.readTemperature();
```

```
    float hum  = dht.readHumidity();
    if (isnan(temp) || isnan(hum)) return;

    // Buat payload JSON
    StaticJsonDocument<200> doc;
    doc["device"]      = CLIENT_ID;
    doc["timestamp"]   = millis() / 1000;
    doc["temperature"] = round(temp * 10) / 10.0;
    doc["humidity"]    = round(hum * 10) / 10.0;
    doc["status"]      = (temp > 30) ? "hot" : "normal";

    char payload[200];
    serializeJson(doc, payload);

    mqtt.publish(TOPIC_PUB, payload);
    Serial.println("[MQTT OUT] " + String(payload));
  }
}
```

E. Percobaan 3: MQTT Subscriber - Kendali Perangkat

Pada percobaan ini, ESP32 akan menerima perintah dari broker MQTT untuk mengendalikan LED/relay. Tambahkan LED pada pin D2 dan modifikasi bagian mqttCallback:

```
// Tambahkan di bagian global:
#define LED_CTRL 2

// Modifikasi fungsi mqttCallback:
void mqttCallback(char* topic, byte* payload, unsigned int length) {
  String msg = "";
  for (int i = 0; i < length; i++) msg += (char)payload[i];
  Serial.println("[CMD] " + msg);

  // Parse perintah JSON
  StaticJsonDocument<100> doc;
  if (!deserializeJson(doc, msg)) {
    String cmd = doc["led"].as<String>();
    if (cmd == "on")  { digitalWrite(LED_CTRL, HIGH); Serial.println("LED ON");
}
    if (cmd == "off") { digitalWrite(LED_CTRL, LOW);  Serial.println("LED
OFF"); }
  }
}

// Contoh payload perintah yang dikirim dari MQTT client:
```

```
// Topic : iot-praktikum/12345/command
// Payload: {"led": "on"}  atau  {"led": "off"}
```

F. Cara Menguji dengan MQTT Client

Tools MQTT Client untuk Pengujian (Gratis)
1. MQTT Explorer: Aplikasi desktop - https://mqtt-explorer.com/ 2. HiveMQ Websocket Client: Browser based - https://www.hivemq.com/demos/websocket-client/ 3. MQTTX: Aplikasi cross-platform - https://mqttx.app/
Cara uji dengan HiveMQ Web Client: • Buka URL di atas, Connect ke broker.hivemq.com • Subscribe ke topic: iot-praktikum/[NIM]/sensor • Amati data JSON yang dikirim oleh ESP32 simulator

G. Tugas Pendahuluan

30. Jelaskan perbedaan protokol MQTT, HTTP REST, dan WebSocket dalam konteks IoT!
31. Apa yang dimaksud dengan QoS (Quality of Service) dalam MQTT? Sebutkan level 0, 1, dan 2!
32. Mengapa JSON lebih populer dibanding XML untuk payload data IoT?

H. Tugas Akhir Praktikum

33. Implementasikan QoS level 1 pada MQTT publish dan subscribe, lalu jelaskan perbedaannya dengan QoS 0
34. Tambahkan fitur 'retained message' pada topik status perangkat
35. Buat dashboard sederhana menggunakan Node-RED atau Grafana Cloud yang menampilkan data dari MQTT broker (opsional - bonus)

MODUL 4

PROYEK AKHIR: SMART HOME MONITORING SYSTEM

A. Tujuan Praktikum

- Mahasiswa dapat mengintegrasikan berbagai komponen IoT dalam satu sistem terintegrasi
- Mahasiswa mampu menerapkan konsep state machine untuk logika kendali IoT
- Mahasiswa dapat merancang arsitektur sistem IoT end-to-end
- Mahasiswa mampu mendokumentasikan dan mempresentasikan proyek IoT

B. Deskripsi Proyek

Pada modul akhir ini, mahasiswa akan membangun sistem Smart Home Monitoring menggunakan ESP32. Sistem ini mengintegrasikan sensor suhu/kelembaban, sensor gerak PIR, LDR (cahaya), dan kendali relay/LED melalui MQTT.

Fitur Sistem Smart Home Monitoring
1. MONITORING LINGKUNGAN • Membaca suhu dan kelembaban (DHT22) setiap 5 detik • Membaca intensitas cahaya (LDR) untuk deteksi siang/malam
• Menampilkan semua data pada LCD 16x2
2. SISTEM KEAMANAN • Sensor PIR untuk deteksi gerakan • Alert via MQTT saat gerakan terdeteksi • LED indikator dan buzzer untuk alarm lokal
3. OTOMASI CERDAS • Lampu otomatis menyala saat gelap (berdasarkan LDR)
• AC virtual (LED indikator) aktif saat suhu > 28°C • Mode 'Away' dan 'Home' dapat diaktifkan via MQTT
4. KONEKTIVITAS IoT • Mengirim semua data sensor ke MQTT broker setiap 10 detik • Menerima perintah kendali dari MQTT (lampu, AC, alarm)

- Status perangkat dikirim sebagai retained message

C. Arsitektur Sistem

Layer	Komponen & Fungsi
Perception Layer	DHT22, PIR, LDR - mengumpulkan data lingkungan
Processing Layer	ESP32 - memproses data, menjalankan logika otomasi
Actuator Layer	LED, Buzzer, Relay (simulasi) - kontrol fisik
Display Layer	LCD 16x2 I2C - tampilan lokal status sistem
Network Layer	WiFi 2.4GHz - koneksi ke internet/cloud
Middleware Layer	MQTT Broker (HiveMQ) - routing pesan IoT
Application Layer	MQTT Dashboard / Mobile App (monitoring & kontrol)

D. Komponen yang Digunakan

Komponen	Pin ESP32 / Fungsi
DHT22 Sensor	Pin D4 — Suhu & Kelembaban
LDR + Resistor 10K	Pin A0 (GPIO34) — Sensor Cahaya
PIR HC-SR501	Pin D15 — Deteksi Gerak
LCD 16x2 I2C	SDA=D21, SCL=D22 — Display
LED Merah (AC)	Pin D26 — Indikator AC Aktif
LED Kuning (Lampu)	Pin D27 — Indikator Lampu Otomatis
LED Merah (Alarm)	Pin D25 — Indikator Alarm Security
Buzzer	Pin D14 — Alarm Bunyi

E. Kode Program Lengkap

```
// ==========================================
// MODUL 4 - Smart Home Monitoring System
// Platform: ESP32 | Simulator: Wokwi
// ==========================================
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>
#include <LiquidCrystal_I2C.h>
#include <ArduinoJson.h>

// ---- PIN DEFINITIONS ----
#define DHT_PIN    4
#define LDR_PIN    34   // ADC1 CH6
```

```
#define PIR_PIN    15
#define BUZZER_PIN 14
#define LED_AC     26
#define LED_LAMP   27
#define LED_ALARM  25
#define DHT_TYPE   DHT22

// ---- THRESHOLDS ----
#define TEMP_THRESHOLD  28.0   // AC aktif di atas suhu ini
#define LIGHT_THRESHOLD 300    // Lampu menyala di bawah nilai ini

// ---- NETWORK CONFIG ----
const char* SSID     = "Wokwi-GUEST";
const char* PASSWORD = "";
const char* BROKER   = "broker.hivemq.com";
const char* BASE_TOPIC = "smarthome/lab/"; // Ubah dengan NIM

// ---- OBJECTS ----
DHT              dht(DHT_PIN, DHT_TYPE);
LiquidCrystal_I2C lcd(0x27, 16, 2);
WiFiClient       wifiClient;
PubSubClient     mqtt(wifiClient);

// ---- SYSTEM STATE ----
struct SmartHomeState {
  float  temperature = 0;
  float  humidity    = 0;
  int    lightLevel  = 0;
  bool   motionDetected = false;
  bool   acActive    = false;
  bool   lampActive  = false;
  bool   alarmActive = false;
  bool   awayMode    = false;
  String lastCommand = "none";
} state;

unsigned long lastSensorRead = 0;
unsigned long lastMQTTPublish = 0;
unsigned long lastLCDUpdate  = 0;

// ---- MQTT CALLBACK ----
void mqttCallback(char* topic, byte* payload, unsigned int len) {
  String msg = "", t = String(topic);
  for (int i = 0; i < len; i++) msg += (char)payload[i];
```

```
  Serial.println("[IN] " + t + " : " + msg);


  StaticJsonDocument<128> doc;
  if (deserializeJson(doc, msg)) return;


  if (t.endsWith("control")) {
    if (doc.containsKey("ac"))    state.acActive    =
(String(doc["ac"].as<String>()) == "on");
    if (doc.containsKey("lamp"))  state.lampActive  =
(String(doc["lamp"].as<String>()) == "on");
    if (doc.containsKey("alarm")) state.alarmActive =
(String(doc["alarm"].as<String>()) == "on");
    if (doc.containsKey("mode"))  state.awayMode    =
(String(doc["mode"].as<String>()) == "away");
    state.lastCommand = msg;
  }
}


// ---- SENSOR READING ----
void readSensors() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  if (!isnan(t) && !isnan(h)) {
    state.temperature = t;
    state.humidity    = h;
  }
  state.lightLevel      = analogRead(LDR_PIN);
  state.motionDetected  = digitalRead(PIR_PIN);
}


// ---- AUTOMATION LOGIC ----
void runAutomation() {
  // Auto AC berdasarkan suhu
  state.acActive = (state.temperature > TEMP_THRESHOLD);


  // Auto lampu berdasarkan cahaya
  state.lampActive = (state.lightLevel < LIGHT_THRESHOLD);


  // Security alarm: PIR aktif saat away mode
  if (state.awayMode && state.motionDetected) {
    state.alarmActive = true;
    Serial.println("[SECURITY] Gerakan terdeteksi saat away mode!");
  }


  // Update aktuator fisik
  digitalWrite(LED_AC,    state.acActive    ? HIGH : LOW);
```

```
  digitalWrite(LED_LAMP,  state.lampActive  ? HIGH : LOW);
  digitalWrite(LED_ALARM, state.alarmActive ? HIGH : LOW);
  if (state.alarmActive) tone(BUZZER_PIN, 2000, 100);
  else noTone(BUZZER_PIN);
}

// ---- LCD DISPLAY ----
void updateLCD() {
  lcd.clear();
  // Baris 1: Suhu dan Kelembaban
  lcd.setCursor(0, 0);
  lcd.print("T:"); lcd.print(state.temperature, 1);
  lcd.print((char)223); lcd.print("C ");
  lcd.print("H:"); lcd.print(state.humidity, 0); lcd.print("%");

  // Baris 2: Status sistem
  lcd.setCursor(0, 1);
  lcd.print(state.acActive   ? "AC:ON " : "AC:OFF");
  lcd.print(state.lampActive ? "L:ON " : "L:OFF");
  lcd.print(state.awayMode   ? "[AWY]" : "[HOM]");
}

// ---- MQTT PUBLISH ----
void publishData() {
  StaticJsonDocument<300> doc;
  doc["temperature"] = state.temperature;
  doc["humidity"]    = state.humidity;
  doc["light"]       = state.lightLevel;
  doc["motion"]      = state.motionDetected;
  doc["ac"]          = state.acActive ? "on" : "off";
  doc["lamp"]        = state.lampActive ? "on" : "off";
  doc["alarm"]       = state.alarmActive ? "on" : "off";
  doc["mode"]        = state.awayMode ? "away" : "home";

  char payload[300];
  serializeJson(doc, payload);
  String topic = String(BASE_TOPIC) + "status";
  mqtt.publish(topic.c_str(), payload, true); // retained=true
  Serial.println("[MQTT] Published: " + String(payload));
}

// ---- SETUP ----
void setup() {
  Serial.begin(115200);
  dht.begin();
```

```
  pinMode(PIR_PIN,    INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(LED_AC,     OUTPUT);
  pinMode(LED_LAMP,   OUTPUT);
  pinMode(LED_ALARM,  OUTPUT);

  lcd.init(); lcd.backlight();
  lcd.setCursor(0,0); lcd.print("Smart Home IoT");
  lcd.setCursor(0,1); lcd.print("  Booting...  ");

  // Connect WiFi
  WiFi.begin(SSID, PASSWORD);
  Serial.print("WiFi connecting");
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println("\nIP: " + WiFi.localIP().toString());

  // Setup MQTT
  mqtt.setServer(BROKER, 1883);
  mqtt.setCallback(mqttCallback);

  lcd.clear();
  lcd.setCursor(0,0); lcd.print("System Ready!");
  delay(1000);
}

// ---- MAIN LOOP ----
void loop() {
  // Maintain MQTT connection
  if (!mqtt.connected()) {
    if (mqtt.connect("SmartHome-ESP32")) {
      mqtt.subscribe((String(BASE_TOPIC) + "control").c_str());
      Serial.println("[MQTT] Connected & subscribed");
    }
  }
  mqtt.loop();

  unsigned long now = millis();

  // Read sensors every 2 seconds
  if (now - lastSensorRead > 2000) {
    readSensors();
    runAutomation();
    lastSensorRead = now;
```

```
  }

  // Update LCD every 1 second
  if (now - lastLCDUpdate > 1000) {
    updateLCD();
    lastLCDUpdate = now;
  }

  // Publish MQTT every 10 seconds
  if (now - lastMQTTPublish > 10000) {
    publishData();
    lastMQTTPublish = now;
  }
}
```

F. Rubrik Penilaian Proyek

Aspek Penilaian	Bobot | Kriteria
Kelengkapan Rangkaian	15% | Semua komponen terhubung dengan benar di Wokwi
Fungsionalitas Sensor	20% | DHT22, LDR, PIR dapat dibaca dengan akurat
Logika Otomasi	20% | AC, lampu, alarm bekerja sesuai threshold
Konektivitas MQTT	25% | Data terkirim dan perintah diterima dengan baik
Kualitas Kode	10% | Terstruktur, terdokumentasi, tanpa bug kritis
Laporan & Presentasi	10% | Diagram sistem, screenshot, analisis data

G. Tugas Pengembangan (Pilih Minimal 2)

36. Tambahkan sensor ultrasonik HC-SR04 untuk memantau level air tangki penampungan
37. Implementasikan Over-the-Air (OTA) firmware update menggunakan ArduinoOTA
38. Integrasikan dengan platform IoT publik: ThingSpeak atau Adafruit IO (keduanya gratis)
39. Buat tampilan dashboard menggunakan Node-RED yang terhubung ke MQTT broker
40. Implementasikan enkripsi MQTT menggunakan TLS/SSL (port 8883) untuk keamanan data

H. Panduan Pembuatan Laporan

Sistematika Laporan

41. Halaman Judul, Tujuan, Dasar Teori
42. Diagram Sistem (blok diagram arsitektur IoT)
43. Skema Rangkaian (screenshot Wokwi)

44. Listing Kode Program dengan penjelasan setiap fungsi
45. Screenshot Hasil Simulasi (LCD, Serial Monitor, MQTT Dashboard)
46. Analisis Data: grafik suhu/kelembaban selama minimal 5 menit simulasi
47. Jawaban Pertanyaan & Tugas Pengembangan
48. Kesimpulan dan Saran Pengembangan

Referensi & Sumber Belajar
Dokumentasi Wokwi : https://docs.wokwi.com Arduino Reference : https://www.arduino.cc/reference/en/ ESP32 Pinout Guide : https://randomnerdtutorials.com/esp32-pinout-reference-gpios/
MQTT Documentation : https://mqtt.org/mqtt-specification/
PubSubClient Lib : https://pubsubclient.knolleary.net/
ArduinoJson Docs : https://arduinojson.org/
HiveMQ Web Client : https://www.hivemq.com/demos/websocket-client/
Node-RED (Gratis) : https://nodered.org/

LAMPIRAN

A. Ringkasan Komponen Wokwi per Modul

Modul	Komponen Utama Wokwi
Modul 1 - Arduino Uno	LED, Resistor, Push Button, Potensiometer
Modul 2 - Arduino Uno	DHT22, LCD I2C 16x2, LDR, Buzzer, LED
Modul 3 - ESP32	DHT22, LED, Buzzer (+ koneksi MQTT virtual)
Modul 4 - ESP32	DHT22, LDR, PIR, LCD I2C, LED x3, Buzzer

B. Cheat Sheet Fungsi Arduino/ESP32

Fungsi	Deskripsi
pinMode(pin, MODE)	Atur pin sebagai INPUT, OUTPUT, atau INPUT_PULLUP
digitalWrite(pin, val)	Set nilai digital pin: HIGH (1) atau LOW (0)
digitalRead(pin)	Baca nilai digital pin: HIGH atau LOW
analogRead(pin)	Baca nilai analog 0-1023 (Arduino) / 0-4095 (ESP32)
analogWrite(pin, val)	Output PWM 0-255 (hanya pin PWM Arduino)
ledcWrite(ch, duty)	PWM ESP32: duty 0-255 pada channel yang dikonfigurasi
delay(ms)	Tunda eksekusi selama ms milidetik (blocking)
millis()	Waktu berjalan sejak boot dalam milidetik (non-blocking)
Serial.begin(baud)	Inisialisasi Serial Monitor dengan baud rate tertentu
Serial.print(val)	Kirim data ke Serial Monitor tanpa newline
map(val, inL, inH, outL, outH)	Peta nilai dari rentang input ke rentang output

C. Troubleshooting Umum

Masalah	Solusi
DHT22 membaca NaN	Pastikan delay(2000) antar pembacaan; cek pin dan resistor pull-up 10K
LCD tidak tampil	Cek alamat I2C (0x27 atau 0x3F); gunakan I2C Scanner sketch
MQTT tidak terhubung	Cek koneksi WiFi terlebih dahulu; coba broker alternatif
ESP32 WiFi gagal	Gunakan SSID 'Wokwi-GUEST' tanpa password di simulator
LED tidak menyala	Cek polaritas LED (anoda ke VCC melalui resistor, katoda ke GND)

Simulasi sangat lambat	Kurangi frekuensi Serial.print; gunakan millis() bukan delay()